SQL avancé (via ORM Django)

https://ricaud.me / @anthony@indieweb.social

Qui a déjà ?

- Ajouté ou modifié un index ?
- Utiliser EXPLAIN ?
- Utilisé Django ?

Plan

- Index Couvrant
- Index Partiel
- Colonnes générées

Contexte

- Je me concentre sur PostgreSQL
- Ce sont des choses que j’ai appliqué
- Ce n’est pas parce que j’en ai parlé qu’il faut l’appliquer partout

Contexte projet

- Projet BetaGouv Docurba
- Procédures de création et de modification des documents d’urbanisme
  - Carte Communale
  - Plan Local d’Urbanisme
  - Schéma de Cohésion Territoriale
- Nous parlerons de commune et de groupements (intercommunalité, pôle métropolitain)

Index Couvrant

Contexte

```
CREATE TABLE commune_adhesions (
    commune_id int NOT NULL,
    groupement_id int NOT NULL
);
```

```
class CommuneAdhesions(models.Model):
    commune = models.ForeignKey(…)
    groupement = models.ForeignKey(…)
```

- On veut des jointures très efficaces

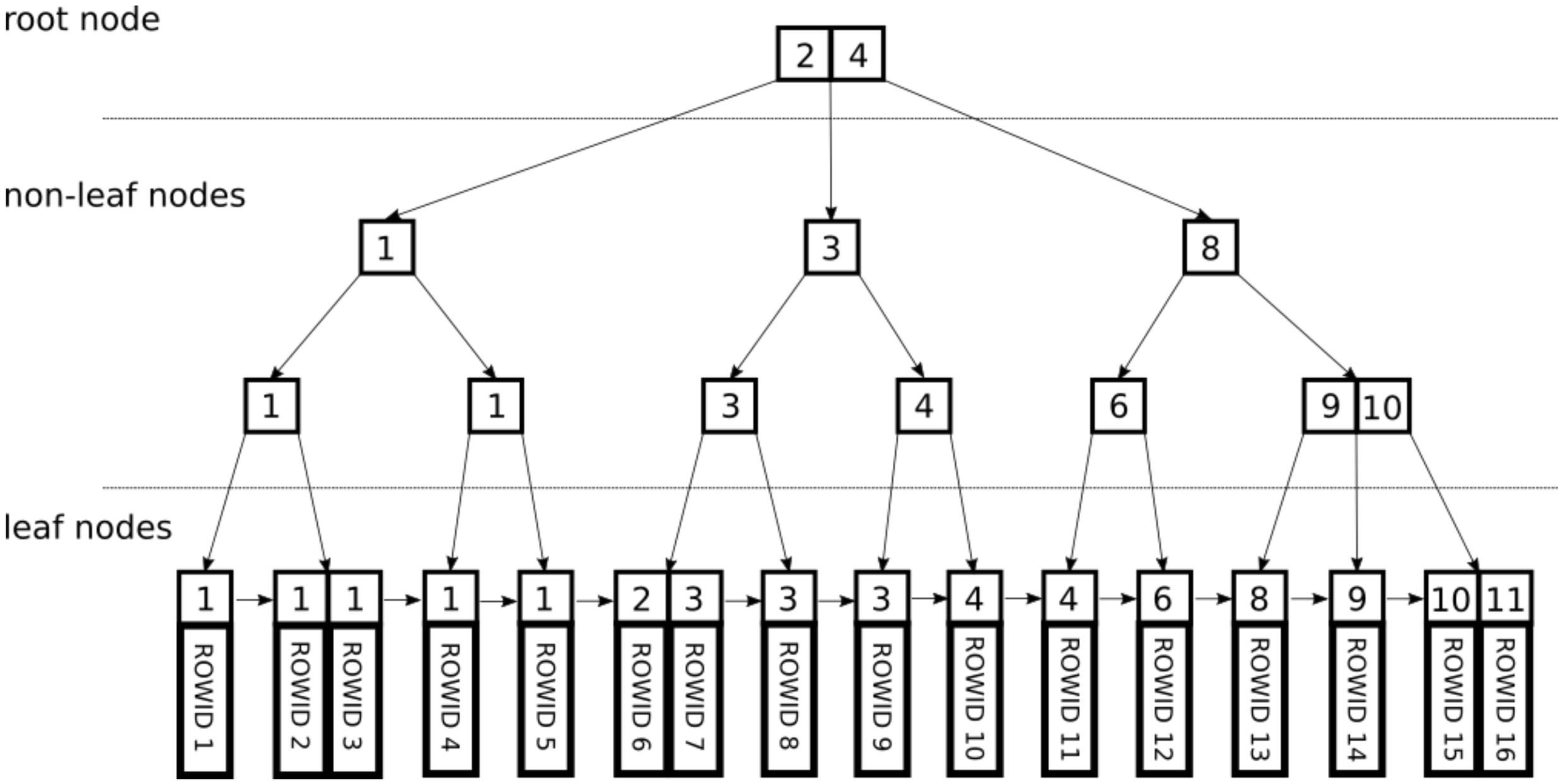
```
SELECT commune_id, groupement_id
FROM commune_adhesions
WHERE commune_id IN (12, 26)
```

Index simple

```
CREATE INDEX commune_adhesions_idx
  ON commune_adhesions
  (commune_id);
```

```
class CommuneAdhesions(models.Model):
    commune = models.ForeignKey(…)
    groupement = models.ForeignKey(…)

    class Meta:
        indexes = [
            models.Index(fields=['commune']),
        ]
```

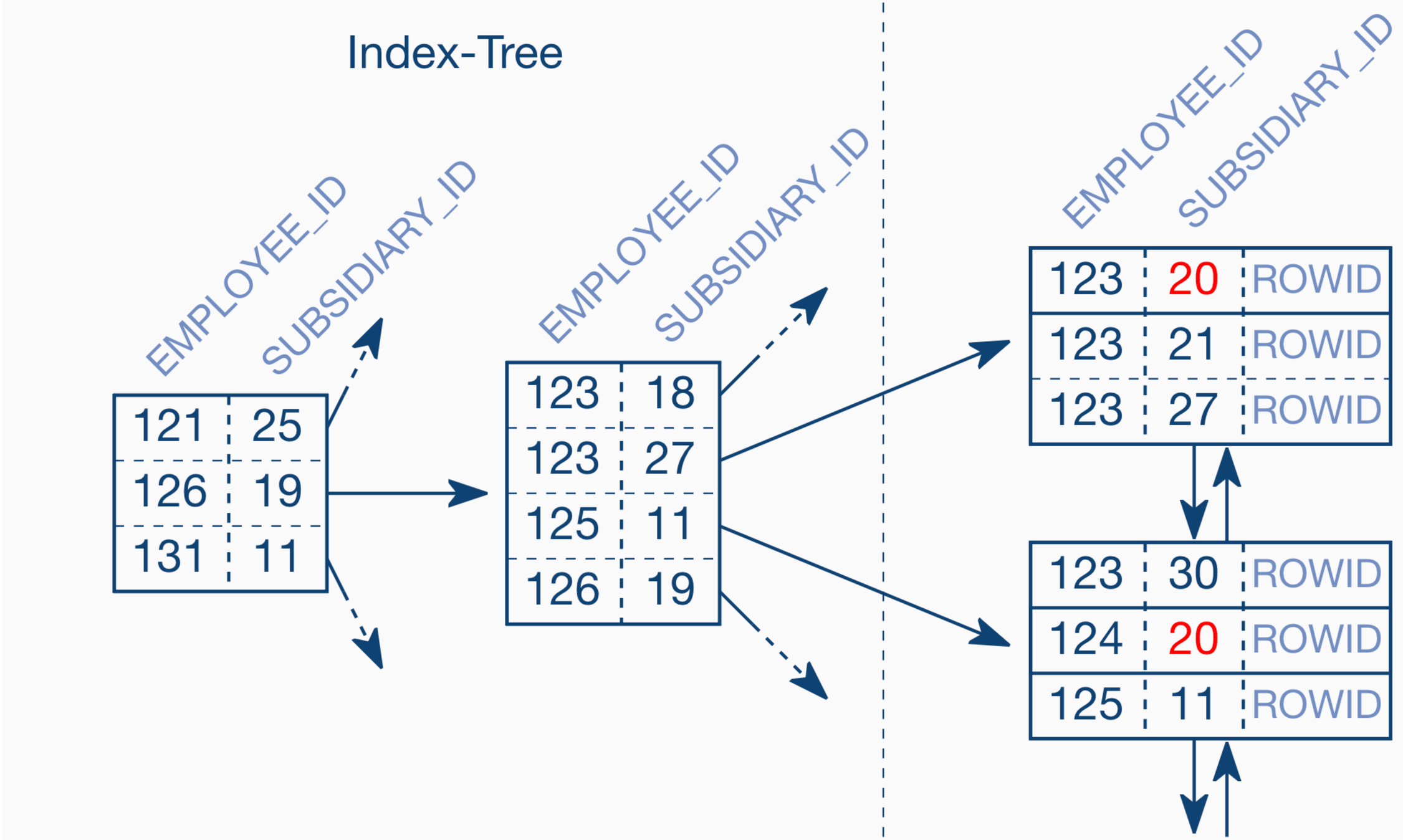

root node
non-leaf nodes
leaf nodes
2
4
1
3
8
1
1
3
4
6
9
10
1
1
1
1
2
3
3
3
4
4
6
8
9
10
11
ROWID 1
ROWID 2
ROWID 3
ROWID 4
ROWID 5
ROWID 6
ROWID 7
ROWID 8
ROWID 9
ROWID 10
ROWID 11
ROWID 12
ROWID 13
ROWID 14
ROWID 15
ROWID 16

Index multi-colonnes

```sql
CREATE INDEX commune_adhesions_idx
  ON commune_adhesions
  (commune_id, groupement_id);
```

```python
class CommuneAdhesions(models.Model):
    commune = models.ForeignKey(…)
    groupement = models.ForeignKey(…)

    class Meta:
        indexes = [
            models.Index(fields=['commune', 'groupement']),
        ]
```


Index-Tree
EMPLOYEE_ID
SUBSIDIARY_ID
121 25
126 19
131 11
EMPLOYEE_ID
SUBSIDIARY_ID
123 18
123 27
125 11
126 19
EMPLOYEE_ID
SUBSIDIARY_ID
123 20 ROWID
123 21 ROWID
123 27 ROWID
123 30 ROWID
124 20 ROWID
125 11 ROWID

Index couvrant

```
CREATE INDEX commune_adhesions_idx
  ON commune_adhesions
  (commune_id) include (groupement_id);
```

```
class CommuneAdhesions(models.Model):
    commune = models.ForeignKey(…)
    groupement = models.ForeignKey(…)

    class Meta:
        indexes = [
            models.Index(fields=['commune'], include=['groupement']),
        ]
```

Impact index couvrant

- Regardons la sortie de EXPLAIN
- Taille de l’index :
  - `\di+ commune_adhesions_idx`
  - Colonne unique : 3.9MB
  - Colonne incluante : 17MB

Index Partiel

Contexte - Fusion de communes

```
CREATE TABLE commune (
    "id" int NOT NULL,
    "nouvelle_id" int,
    CONSTRAINT "commune_nouvelle_id_fkey" FOREIGN KEY ("nouvelle_id")
REFERENCES commune("id"),
    PRIMARY KEY ("id")
);
```

```
class Commune(models.Model):
    nouvelle = models.ForeignKey(
        "self", models.DO_NOTHING, null=True, related_name="deleguee"
    )
```

Index simple

```
CREATE INDEX commune_idx
  ON commune (id);
```

```
class Commune(models.Model):
    nouvelle = models.ForeignKey(…)

    class Meta:
        indexes = [
            models.Index(fields=['id']),
        ]
```

Index partiel

```
CREATE INDEX commune_nouvelle_not_null_idx
  ON commune (id)
  WHERE (nouvelle_id IS NOT NULL);
```

```
class Commune(models.Model):
    nouvelle = models.ForeignKey(…)

    class Meta:
        indexes = (
            models.Index(
                fields=["id"],
                condition=models.Q(nouvelle__isnull=False),
            ),
        )
```

Impact index partiel

- Taille de l’index :
  - `\di+ commune_nouvelle_not_null_idx`
  - Index simple : 1168KB
  - Index partiel : 96KB

Colonnes générées

Contexte

- Ré-écriture itérative pour éviter un long feature freeze
  - Stack existante (pas en Django)
  - Stack en création avec Django
- Les deux sont en productions
- Stack existante n’a pas de tests automatisés
- Modélisation des données laisse à désirer

Existant

```
class Procedure(models.Model):
    archived = models.BooleanField(db_default=False)
```

Suivons les duplicata

```
class Procedure(models.Model):
    archived = models.BooleanField(db_default=False)
    duplicate_of = models.OneToOneField(
        "self",
        on_delete=models.DO_NOTHING,
        null=True,
    )
```

Renommons

```
class Procedure(models.Model):
    soft_delete = models.BooleanField(db_default=False)
    duplicate_of = models.OneToOneField(
        "self",
        on_delete=models.DO_NOTHING,
        null=True,
    )
```

Retour de archived

```
class Procedure(models.Model):
    soft_delete = models.BooleanField(db_default=False)
    duplicate_of = models.OneToOneField(…)
    archived = models.GeneratedField(
        output_field=models.BooleanField(),
        db_persist=True,
        expression=(
            models.Q(soft_delete=True) |
            models.Q(duplicate_of__isnull=False)
        ),
    )
```

SQL

```
CREATE TABLE public.procedures (
  doublon_cache_de_id uuid,
  soft_delete boolean DEFAULT false NOT NULL,
  archived boolean
    GENERATED ALWAYS AS (
      doublon_cache_de_id IS NOT NULL OR soft_delete
    ) STORED
);
```

Resources Index

- https://docs.djangoproject.com/en/6.0/ref/models/indexes/
- https://www.postgresql.org/docs/current/indexes-index-only-scans.html
- https://hakibenita.com/
- https://use-the-index-luke.com/
- https://explain.dalibo.com/

Resources Colonnes générées

- https://docs.djangoproject.com/en/6.0/ref/models/fields/#generatedfield
- https://www.postgresql.org/docs/current/ddl-generated-columns.html
- https://www.paulox.net/2023/11/24/database-generated-columns-part-2-django-and-postgresql/

SQL avancé (via ORM Django)

https://ricaud.me / @anthony@indieweb.social